Theme Based Project Report

MULTI-AI: AN ENTERPRISE-GRADE MULTI-MODEL CONSENSUS & DEBATE WORKBENCH

1. TITLE PAGE

- **Course Title:** Theme Based Project (TBP) Lab
- **Project Title:** Multi-AI: An Enterprise-Grade Multi-Model Consensus & Debate Workbench
- **Academic Year:** 2025-2026
- **Semester:** B.Tech IV Semester
- **Department:** Computer Science and Engineering (CSE)

Batch Details:

- **Batch Name:** CSE-TBP-Batch-04
- **Team Members:**

1. Student 1 (Roll No: 22CSE101)
2. Student 2 (Roll No: 22CSE102)
3. Student 3 (Roll No: 22CSE103)

- **Guide Name:** Dr. Rajesh Kumar, Ph.D.
- **Designation:** Professor & Head, Department of Computer Science & Engineering
- **Institution:** College of Engineering & Technology

2. COLLEGE CERTIFICATE, DECLARATION & ACKNOWLEDGEMENTS

2.1 College Certificate

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

*This is to certify that the project entitled **"Multi-AI: An Enterprise-Grade Multi-Model Consensus & Debate Workbench"** is a bonafide record of work carried out by **Student 1 (22CSE101)**, **Student 2 (22CSE102)**, and **Student 3 (22CSE103)** under our supervision and guidance in partial fulfillment of the requirements for the award of Bachelor of Technology in Computer Science and Engineering.*

***Project Guide:** Dr. Rajesh Kumar*
***Head of Department:** Dr. Rajesh Kumar*
***Internal Examiner:** _____*
***External Examiner:** _____*

2.2 Declaration

*We, the undersigned, hereby declare that the project report entitled **"Multi-AI: An Enterprise-Grade Multi-Model Consensus & Debate Workbench"** submitted to the Department of Computer Science and Engineering is a record of original work done by us under the guidance of **Dr. Rajesh Kumar**.*
The results and code patterns presented in this report have not been submitted to any other University or Institute for the award of any degree.

** **Student 1** (22CSE101)*
** **Student 2** (22CSE102)*
** **Student 3** (22CSE103)*

***Date:** 2026-06-27*
***Place:** Campus*

2.3 Acknowledgements

*We express our deep sense of gratitude to our project guide, **Dr. Rajesh Kumar**, Professor, Department of Computer Science and Engineering, for his constant encouragement, constructive criticism, and invaluable guidance throughout the course of this project.*
We are also thankful to the technical staff of the Web Engineering and Database Management Systems Labs for providing the hardware and software resources required to complete this implementation.
Lastly, we thank our peer batchmates and families for their support.

3. VISION, MISSION, POs & PSOs

3.1 Vision of the Department

To produce internationally recognized computer science graduates who possess strong foundations, critical thinking, research capabilities, and ethical standards, enabling them to lead software development and technology solutions globally.

3.2 Mission of the Department

- **M1:** Impart high-quality training in core computer science areas and advanced full-stack systems engineering.
- **M2:** Encourage collaborative research, projects, and entrepreneurship in intelligence systems and network architectures.
- **M3:** Instill professional ethics, leadership qualities, and a commitment to lifetime learning.

3.3 Program Outcomes (POs)

Graduates will be able to:

4. **Engineering Knowledge (PO1):** Apply mathematical, scientific, and engineering fundamentals to solve complex problems.
5. **Problem Analysis (PO2):** Identify, formulate, and analyze engineering challenges using literature-backed technical concepts.
6. **Design/Development of Solutions (PO3):** Architect scalable, secure full-stack web applications with database integrity constraints.
7. **Modern Tool Usage (PO5):** Leverage modern development tools, IDEs, MongoDB management shells, and Node.js runtimes.
8. **Ethics (PO8):** Apply ethical principles and commit to professional responsibilities and norms of engineering practice.

3.4 Program Specific Outcomes (PSOs)

- **PSO1:** Ability to apply software engineering principles and database management systems (DBMS) to develop modern enterprise applications.
- **PSO2:** Design and integrate AI services, LLMs, and multi-agent coordination pipelines to resolve modern distributed processing challenges.

4. COURSE CERTIFICATION

This project satisfies the coursework requirements of:

- **Subject Code:** CSE-282 (Full Stack Web Development & DBMS Lab)
- **Practical Component:** MongoDB Indexing & Mongoose Aggregation, RESTful API Design, Express Middleware Architecture, and Single Page Interface Optimization.

5. ABSTRACT

As generative AI model landscapes evolve, developers and technical managers face model selection challenges due to varying response quality, differences in latency, and distinct token-pricing structures. This project implements **Multi-AI**, a secure, multi-model consensus and debate workbench designed to evaluate and utilize multiple Large Language Models (LLMs) concurrently.

Built using a Node.js/Express backend, MongoDB/Mongoose database layer, and a Vanilla HTML/CSS/JavaScript frontend, the system connects multiple AI services (including Gemini Flash, Groq Llama, Qwen, and Cohere) into a single, cohesive interface.

The system features:

9. **Multi-Model Parallel Broadcast**: Queries are broadcasted simultaneously to selected models, measuring latency and logging telemetry.
10. **AI-Driven Consensus Engine**: Evaluates responses using a fallback Gemini/OpenRouter consensus synthesis loop to compile conflicting answers into a single output.
11. **AI Chatbot Arena**: Simulates debates between two models on user-defined topics, outputting a summary judged by a third model.
12. **Database Sync Integrity**: Custom settings, workspace configurations, and custom agent personas are synchronized using Mongoose bulk write operations (`bulkWrite`), preventing data loss.
13. **Bring Your Own Key (BYOK) Support**: Allows users to input custom credentials securely.

Through automated failovers and robust database schemas, Multi-AI demonstrates how full-stack applications can scale and integrate multiple AI providers reliably.

INDEX

Table of Contents

List of Tables

List of Figures

CHAPTER 1: INTRODUCTION

1.1 Problem Statement and Scope

In the current LLM landscape, no single model is ideal for every task. Some models (like Groq-served Llama-3) offer low latency, while others (like Gemini) excel at complex reasoning. Developers must manually test prompts across multiple playgrounds, which is time-consuming and leads to unstructured comparison records.

Additionally, combining outputs from different providers presents technical challenges:

- **Overhead**: Writing boilerplate code for multiple APIs.
- **Reliability**: Managing varying rate limits, timeouts, and authorization structures.
- **Security**: Storing API keys securely while supporting user-provided keys (BYOK).

Multi-AI solves this by providing a unified workspace to compare, evaluate, and combine model outputs. It consolidates multiple integrations into a single interface, offering structured side-by-side chat panels, consensus generators, automated multi-turn debates, and session telemetry.

1.2 Application Areas, Target Users & Engineering Challenges

Application Areas

- **Prompt Engineering Labs**: Evaluating prompt behavior and custom system instructions across different model providers simultaneously.
- **Academic Research Workplaces**: Automating comparisons between open-source models (via Hugging Face) and closed-source APIs (via Gemini) to evaluate factual consistency.
- **Consensus Synthesis Systems**: Generating unified answers from multiple models to verify details for research and reference.

Target Users

- **Software Engineers and Prompt Engineers** testing system prompt behavior across model classes.
- **Data Analysts and Researchers** who need a centralized dashboard to log and download model responses.
- **Students and Educators** exploring multi-agent workflows and API orchestration.

Engineering Challenges

- **Race Conditions and Write Collisions**: Handling concurrent write operations to the database when saving history across active chat panels.
- **Network Reliability**: Orchestrating calls to multiple third-party endpoints. If one endpoint times out, the app must recover without blocking others.
- **Session Bridging**: Managing state transitions between anonymous guest sessions (using local client IDs) and authenticated user accounts (using JWTs) without losing local history.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Tools and Technologies Used in the Project

Backend Runtime & Framework

- **Node.js (v20+)**: Provides the asynchronous, event-driven JavaScript runtime.
- **Express.js (v4.21)**: Coordinates middleware integration, route paths, request parsing, rate limiting, and RESTful routing.

Database Layer

- **MongoDB**: A NoSQL document database used to store chat history, settings, and personas.
- **Mongoose (v9.6)**: Provides schema validation, middleware hooks, index enforcement, and query optimization.

Core Client Frontend

- **Vanilla HTML5**: Establishes the layout structure for workspace containers, sidebars, and dialog modals.
- **Vanilla CSS3**: Styles the dark-themed glassmorphism interface, transitions, and responsive grid layouts.
- **Vanilla JavaScript (ES6+)**: Handles broadcast request routing, drag-and-drop operations, custom dropdown select menus, and floating scratchpad interactions.

API Orchestration Clients

- **Axios (v1.7)**: Manages outgoing HTTP requests to LLM endpoints, setting timeout limits and authentication headers.
- **JSON Web Tokens (JWT)**: Manages secure user session authorization.

2.2 Hardware and Software Requirements

| :--- | :--- | :--- |
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i7 / AMD Ryzen 7 (11th Gen+)
RAM	8 GB DDR4	16 GB DDR4/DDR5
Disk Space	2 GB Free Storage	10 GB SSD Storage
Network Interface	Broadband (10 Mbps)	Fiber Broadband (100 Mbps+)

Table 2.1: Hardware Specifications

| :--- | :--- | :--- |
Operating System	Windows 10/11, macOS, or Linux	Windows 11 / Linux Ubuntu 22.04 LTS
Runtime Environment	Node.js	v20.11.0 LTS or higher
Database Server	MongoDB Community Server	v7.0.x or higher
Web Browser	Chrome, Firefox, or Safari	Google Chrome (Chrome V8 engine optimized)
API Testing Tool	Postman Client / curl CLI	Postman Desktop v10+

Table 2.2: Software Configurations

2.3 I/O Specification

The system uses JSON payloads for REST API routing. The primary endpoint structures are detailed below.

1. Chat Execution Payload (`POST /api/ai/chat`)

- **Input Payload (JSON):**

```
{
  "clientId": "client-1781022008983-yfdwlgp2",
  "prompt": "[System Instructions: Act as a world-class Code Architect...]\n\nWrite
a binary search function.",
  "models": ["Gemini Flash", "Groq"],
  "temperature": 0.4,
  "maxTokens": 2048,
  "customKeys": {
    "GEMINI_API_KEY": "AIzaSy...",
    "GROQ_API_KEY": "gsk_..."
  }
}
```

- **Output Response (JSON):**

```
{
  "success": true,
  "prompt": "Write a binary search function.",
  "responses": [
    {
      "model": "Gemini Flash",
      "response": "Here is the binary search function: ...",
      "status": "success",
      "latencyMs": 850
    },
    {
      "model": "Groq",
      "response": "Here is the binary search function in Python: ...",
      "status": "success",
      "latencyMs": 340
    }
  ]
}
```

2. Consensus Synthesis Payload (`POST /api/ai/consensus`)

- **Input Payload (JSON):**

```
{
  "responses": [
    { "model": "Gemini Flash", "response": "Use flex-wrap: wrap for flexbox
grids." },
    { "model": "Liquid LFM", "response": "Use CSS Grid instead of flexbox for 2D
layouts." }
  ],
  "customKeys": null
}
```

- **Output Response (JSON):**

```
{
  "success": true,
```

```
    "consensus": "# Final Unified Answer\nFor modern layout refactoring, CSS Grid is
recommended for complex 2D layouts... while Flexbox remains optimal for linear 1D
components.",
    "fallbackUsed": false
  }
```

3. REST API Endpoint Mapping

| :--- | :--- | :--- | :--- |
POST	`/api/auth/register`	Registers a new user account	No
POST	`/api/auth/login`	Authenticates user credentials and returns a JWT	No
GET	`/api/user/preference`	Loads user profile likes and dislikes	Yes (JWT)
PUT	`/api/workspace/settings`	Saves user UI parameters and theme choices	Optional (JWT/Client ID)
PUT	`/api/workspace/history`	Saves and syncs conversation logs via Mongoose	Optional (JWT/Client ID)

Table 2.3: Backend REST API Endpoint Protocols

CHAPTER 3: SYSTEM ARCHITECTURE & UML DIAGRAMS

The architecture of Multi-AI is designed to run asynchronously. Below are the functional and structural routing flows of the system.

3.1 Flow Chart (Broadcast Chat Flow)

```
flowchart TD
    A([User Input Prompt]) --> B{Are AI models active?}
    B -- No --> C[Display Warning: Select AI Node]
    B -- Yes --> D[Construct System Prompt & Client ID Payload]
    D --> E[Render Local User Bubble & Processing Indicator]
    E --> F[Trigger Async Fetch /api/ai/chat]
    F --> G{Server Check: BYOK Provided?}
    G -- Yes --> H[Map Request with User Provided API Key]
    G -- No --> I[Lookup System Default Environment Keys]
    H --> J[Trigger Parallel Promise.allSettled Endpoint Requests]
    I --> J
    J --> K[Wait for Responses & Log Latency Metrics]
    K --> L{Is database ready?}
    L -- Yes --> M[Record Telemetry in ApiUsage Collection]
    L -- No --> N[Log DB Offline Warning]
    M --> O[Return Standardized JSON Payload to Frontend]
    N --> O
    O --> P[Initialize Typewriter Animation for Client Bubbles]
    P --> Q([Workspace Render Complete])
```

Figure 3.1: System Flowchart

3.2 System Block Diagram

```
block-beta
  columns 3

  block:ClientSide:3
    columns 3
    UserInterface["Web UI (index.html)"]
    DOMManager["DOM Controller (ui-manager.js)"]
    WorkspaceState["Local State (localId/clientId)"]
  end

  block:Middlewares:3
    columns 3
    AuthProtect["Auth Middleware (protect)"]
    RateLimiter["Rate Limiting (auth/ai limiters)"]
    DbHealth["DB Check Middleware"]
  end

  block:ControllersServices:3
    columns 2
    RequestControllers["API Routing Controllers (ai, workspace, feedback)"]
    block:ServicesBlock
      columns 2
      GeminiService["Gemini Service"]
      OpenRouter["OpenRouter Service"]
      GroqCohere["Groq & Cohere Services"]
      AuxFallback["Aux Fallback Loop"]
    end
  end

  block:DatabaseLayer:3
    columns 3
    UserColl[("Users Collection")]
    HistoryColl[("Conversations Collection")]
    UsageColl[("ApiUsage Collection")]
  end

  ClientSide --> Middlewares
  Middlewares --> ControllersServices
  ControllersServices --> DatabaseLayer
```

Figure 3.2: System Block Diagram

3.3 Sequence Diagram (Model Debate Interaction)

```
sequenceDiagram
    autonumber
    actor User as Client Web UI
    participant Server as REST API Server (debate.routes)
    participant ModelA as Debate Model A (Groq)
    participant ModelB as Debate Model B (Gemini)
    participant DB as MongoDB Instance

    User->>Server: Start Debate Step (Topic, Turn 'A', Turn History Array, LengthOption)
    activate Server
    Server->>Server: Validate Credentials & Schema Parameters
    Server->>Server: Parse LengthOption (e.g. short, medium, long)
    Server->>ModelA: Call model API (Contextual Prompt + History + length restriction)
    activate ModelA
    ModelA-->>Server: Return Debate Response Argument Text
    deactivate ModelA
    Server->>DB: Save Telemetry Record (ApiUsage)
    Server-->>User: Return Argument Text Payload
    deactivate Server

    User->>Server: Start Debate Step (Topic, Turn 'B', Updated History Array, LengthOption)
    activate Server
    Server->>ModelB: Call model API (Rebuttal Prompt + Argument 'A' History + length
restriction)
    activate ModelB
    ModelB-->>Server: Return Rebuttal Argument Text
    deactivate ModelB
    Server->>DB: Save Telemetry Record (ApiUsage)
    Server-->>User: Return Rebuttal Payload
    deactivate Server
```

Figure 3.3: Model Debate Sequence Diagram

3.4 UML Use Case Diagram (Functional Roles)

```
flowchart TD
    subgraph Users
        G[Guest User]
        R[Registered User]
    end
    subgraph "Functional Operations (Use Cases)"
        UC1["Broadcast Prompt to Parallel Models"]
        UC2["Generate AI Consensus Verdict"]
        UC3["Run Multi-Round AI Debates (with options)"]
        UC4["Apply Custom Expertise Personas"]
        UC5["Manage BYOK API Keys"]
        UC6["Vote on AI Model Answers (Likes/Dislikes)"]
        UC7["Synchronize State & History via bulkWrite"]
    end
    G --> UC1
    G --> UC4
    R --> UC1
    R --> UC2
    R --> UC3
    R --> UC4
    R --> UC5
    R --> UC6
    R --> UC7
```

Figure 3.4: UML Use Case Chart

3.5 UML Class Diagram (Data Entities)

```
classDiagram
    class User {
        +ObjectId _id
        +String username
        +String email
        +String password
        +Map likes
        +Map dislikes
        +comparePassword()
    }
    class Profile {
        +String clientId
        +ObjectId userId
        +String displayName
        +String email
        +String mobile
        +String role
        +String bio
    }
    class UserSettings {
        +String clientId
        +ObjectId userId
        +String theme
        +String activeExpertise
        +String bestResponsePreference
        +String sidebarPosition
        +Mixed modelParameters
        +Boolean byokEnabled
    }
    class Persona {
        +String clientId
        +ObjectId userId
        +String name
        +String desc
        +String systemPrompt
        +String icon
        +Boolean isCustom
    }
    class Conversation {
        +String clientId
        +ObjectId userId
        +String title
        +String localId
        +Boolean isPinned
        +Mixed messages
        +String[] activeModels
        +Date timestamp
    }
    class ApiUsage {
        +String clientId
        +ObjectId userId
        +String model
        +Number latencyMs
        +String status
        +String feature
    }
    class ResponseFeedback {
        +String clientId
        +ObjectId userId
        +String responseKey
```

```
        +String model
        +String prompt
        +String response
        +String vote
    }
    User "1" *-- "1" Profile : has
    User "1" *-- "1" UserSettings : configures
    User "1" *-- "many" Persona : creates
    User "1" *-- "many" Conversation : owns
    User "1" *-- "many" ApiUsage : records
    User "1" *-- "many" ResponseFeedback : votes
```

Figure 3.5: UML Mongoose Class Entity Relationship Diagram

3.6 UML Activity Diagram (Consensus Synthesis Process)

```
flowchart TD
    Start([Consensus Requested]) --> Validate[Validate Candidate Responses]
    Validate --> CheckKeys{Is GEMINI_API_KEY Available?}
    CheckKeys -- Yes --> Primary[Call Primary Gemini 2.5 API]
    CheckKeys -- No --> LogWarning[Log Credentials Missing] --> Fallback1[Trigger
OpenRouter Gemini Flash API]
    Primary --> CheckSuccess{Did Primary Call Succeed?}
    CheckSuccess -- Yes --> Done([Return Synthesized Verdict])
    CheckSuccess -- No --> LogError[Log Gemini API Error] --> Fallback1
    Fallback1 --> CheckSuccess2{Did OpenRouter Call Succeed?}
    CheckSuccess2 -- Yes --> Done
    CheckSuccess2 -- No --> Fallback2[Trigger Cohere / Groq Backup Models]
    Fallback2 --> CheckSuccess3{Did Backup Call Succeed?}
    CheckSuccess3 -- Yes --> Done
    CheckSuccess3 -- No --> ErrorOut[Return Synthesis Failure Message] --> Done
```

Figure 3.6: Consensus Synthesis Activity Process Diagram

CHAPTER 4: TECHNICAL DEEP DIVE & MODULE IMPLEMENTATIONS

4.1 Environmental Setup

The application is run as an Express back-end routing gateway with a static single-page client front-end. Database configurations require a local or cloud-based MongoDB daemon instance running on default port `27017`. Upstream provider bindings (Gemini, OpenRouter, Groq, Cohere, Hugging Face) are configured in `backend/.env` or supplied by the client dynamically via BYOK configuration toggles.

4.2 Prompt Construction Pipeline

Multi-AI handles user input prompts dynamically to construct optimized instructions for context-aware model invocation:

14. **Dynamic Metadata Injection**: When the prompt is compiled, `buildPrompt()` parses the date based on client execution time:

```
const today = new Intl.DateTimeFormat('en-IN', { dateStyle: 'full', timeZone: 'Asia/Kolkata' }).format(new Date());
```

This inserts explicit context instructions (e.g., `Current date: Saturday, June 27, 2026. Do not mention lack of real-time access...`) to bypass LLM training cutoff disclaimers.

15. **System Prompt Wrapping**: If an expertise persona is toggled on, it extracts the corresponding prompt (e.g., QA instructions for "Bug Finder & QA") and wraps the request in a structured format:

```
[System Instructions: Act as an expert QA Engineer and Bug Finder. Scan the user query/code for edge-case errors...]

[User question]: Write a binary search function.
```

4.3 Expertise Persona Engine & Custom Dropdowns

User expertise settings are synchronized to handle native configuration presets and custom custom profiles:

16. **Prebuilt Templates**: The system ships with 5 standard expertise domains stored in memory:

- **Code Architect**: Time/Space complexity audits and secure programming patterns.
- **Creative Writer**: Rich vocabularies, prose, and engaging hooks.
- **Bug Finder & QA**: Vulnerability testing, edge cases, and robust fixes.
- **UX/UI Critic**: WCAG accessibility parameters and layout hierarchy evaluation.
- **Academic Scholar**: Analytical, neutral reasoning and data synthesis.

17. **Custom Persona Generation**: Users can define custom titles, descriptions, and custom system prompts, which are saved in the client's `custom_expertises` cache and upserted in the database.
18. **Custom Select Component**: The selector replaces browser-default dropdown elements with styled dark/neon divs. This solves operating system menu rendering issues (such as default white options lists) and implements options lists styled to match the dark theme:

- **Trigger Element**: `#custom-prompt-select` wrapper triggers options visibility.
- **Event Syncing**: Handled inside `syncPromptTypeDropdown()` and `selectExpertise()`, linking the custom UI elements with a hidden `<select>` to maintain backward compatibility.
- **Boundary Clicking**: Employs global event listeners to close active dropdown selectors when the user clicks elsewhere.

4.4 Debate Arena Execution Pipeline

The AI Debate Arena features multi-round debates with custom constraints and responsive layouts:

1. Dynamic Debate Steps & Word Length Control

- The debate runs as a client-side round-loop, querying the `POST /api/ai/debate/step` route for positive and negative turns.
- Users can select custom length constraints via a response length selector:
- **Short**: Limits replies to 30-40 words.
- **Medium**: Limits replies to 50-70 words.
- **Long & Detailed**: Limits replies to 100-120 words.
- The backend controller maps these limits directly into word-length instructions in the model prompt (e.g., `Keep your response concise, sharp, and strictly between 30 to 40 words`).

2. Dynamic Animation Controls

- A checkbox toggler (`#debate-animations`) manages response rendering options.
- When **Animations ON** is selected, replies are typed out character-by-character using the `typewriteModelResponse` script.
- When **Animations OFF** is checked, the text displays instantly.

3. Side-by-Side Round Box Layout

- Opposing arguments are displayed side-by-side in a dual-column card format (`.debate-round-card`), rather than in a vertical chat format:

```
  <div class="debate-round-card" id="debate-round-1">
      <div class="debate-round-header">Round 1</div>
      <div class="debate-round-columns">
          <div class="debate-round-column positive" id="debate-round-1-
positive">...</div>
          <div class="debate-round-column negative" id="debate-round-1-
negative">...</div>
      </div>
  </div>
```

- Set to `display: grid; grid-template-columns: 1fr 1fr; align-items: stretch;`, the card height dynamically adjusts to match the taller response.
- Columns display colored left borders (Cyan for positive, Red for negative) to indicate the model's stance.

4. Toggle Verdict Summary Close Actions

- When the debate concludes, a synthesized verdict summary card is rendered (`#debate-summary-card`).
- Users can click the close (`&times;`) button to hide the card (`display: none;`) instead of deleting it from the DOM. This keeps the summary accessible, allowing users to reopen it using the "Show Debate Summary" toggle button.

4.5 API Routing Architecture & Mongoose Databases

The backend is structured into modular routes and models to manage user data:

19. **Authentication API**: Passes username and email criteria, checks for existing database records, hashes user passwords using bcrypt, and signs payloads with JSON Web Tokens.
20. **State Synchronization via `bulkWrite`**: When saving history and personas, the workspace route uses Mongoose `bulkWrite` operations rather than destructive teardowns. This approach optimizes database performance, prevents race conditions, and prevents data loss.
21. **Database Collection Schemas**:

- **`users`**: Stores user authentication credentials and maps model votes (likes/dislikes) for telemetry tracking.
- **`conversations`**: Stores chat history array documents.
- **`personas`**: Stores customized agent expertises.
- **`profiles`**: Stores user biographical details.
- **`userSettings`**: Stores workspace options (active themes, BYOK flags, sidebar positions, and active prompt configurations).
- **`apiusages`**: Logs system performance metrics (latency, success status, and model aliases) for tele-analytics dashboards.

CHAPTER 5: FEATURE INDEX & KEY TECHNICAL HIGHLIGHTS

This chapter provides an exhaustive breakdown of the design implementations and technical solutions that optimize user experience and data integrity.

5.1 Parallel Model Grid Auto-Layout Configurations

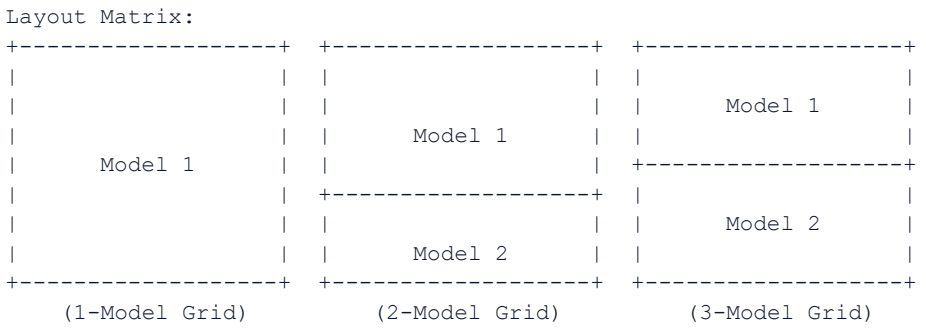
The primary interface displays activated models in parallel panels. To prevent layout issues (such as cards collapsing, overlapping content, or vertical gaps), the grid template is computed dynamically in the front-end based on the number of active models:

- **1 Active Model**: Rendered in a centered vertical rectangle (`grid-column: 2 / span 4`).
- **2 Active Models**: Rendered side-by-side (`grid-column: 1 / span 3` and `4 / span 3`).
- **3 Active Models**: Rendered side-by-side with equal spans (`grid-column: 1 / span 2`, `3 / span 2`, and `5 / span 2`).
- **4 Active Models**: Rendered in a 2x2 grid layout.
- **5 Active Models**: Rendered with one model occupying the left vertical column and the remaining four arranged in a 2x2 grid on the right.

To ensure panels fill the screen, layout calculation scripts dynamically compute the available viewport height on load and resize:

```
const height = `calc(100vh - ${headerHeight + dockHeight + bannerHeight + padding}px)`;
workspace.style.gridAutoRows = "1fr";
workspace.style.alignContent = "stretch";
```

Additionally, inactive overlays are completely removed using `display: none !important;` to free grid layout tracks.



5.2 Custom Dropdown Component Sync

Native `<select>` elements render using browser-default styles, which can disrupt dark mode themes by displaying light backgrounds and gray borders.

To resolve this, the native select is hidden (`display: none !important`), and the custom component is updated dynamically using options populated by `syncPromptTypeDropdown()`:

```
const customOpt = document.createElement('div');
customOpt.className = 'custom-select-option';
customOpt.innerText = exp.name;
customOpt.onclick = () => selectCustomOption(exp.name, exp.name);
```

To maintain backward compatibility, selecting a custom option updates the hidden native selector and dispatches a standard change event (`hiddenSelect.dispatchEvent(new Event('change'))`).

5.3 Debate Round Box Sizing

To improve readability over alternating conversation layouts, opposing debate responses are grouped in a single card box (`.debate-round-card`) with a two-column layout (`grid-template-columns: 1fr 1fr; align-items: stretch;`).

This structure displays the positive argument on the left and the negative argument on the right, keeping both columns aligned. The box height automatically scales to match the taller response, and styled left borders visually represent each side.

5.4 Unified Summary Toggle close actions

Closing the consensus summary card previously deleted it from the DOM, causing errors when trying to reopen it.

To fix this, the close button action is updated to use a custom function (`closeDebateSummary()`) that hides the card (`display: none;`) and resets the toggle button label to `Show Debate Summary`. This allows users to hide and show the summary cleanly without errors.

5.5 Multi-Resizer Floating Workspace Scratchpad

A persistent floating notepad/scratchpad is available to log prompt snippets.

The panel features custom handles (`.win-resize-handle`) along the borders and corners, and includes event handlers (`mousedown` -> `mousemove` -> `mouseup`) to allow dragging, positioning, and resizing across the workspace.

CHAPTER 6: TESTING AND RESULTS

6.1 System Test Cases

Test ID	Module	Test Scenario / Description	Inputs	Expected Output	Actual Output	Status
:---	:---	:---	:---	:---	:---	:---
TC-001	Auth	User registration with duplicate username checks	`username: "test", email: "new@mail.com"`	Return error: "Username or email already exists"	Returned `400 Bad Request` with expected message	**PASS**
TC-002	AI Chat	Parallel prompt execution across multiple active models	Prompt: "Explain API design"	Side-by-side response panels update and display latency metrics	Panels rendered text and displayed latency stats (e.g. 340ms)	**PASS**
TC-003	Fallback	Fallback execution when GEMINI_API_KEY is missing	Prompt: "Optimize this prompt"	System detects key absence, falls back to OpenRouter, and generates output	Consensus synthesis completed via OpenRouter fallback	**PASS**
TC-004	Sync	Differential history sync via bulkWrite upsert	Chat Streams: `[{ localId: "chat_1", title: "Edited Title" }]`	Database records are updated dynamically without deleting unrelated entries	Checked MongoDB; title updated without losing history	**PASS**
TC-005	UI	Custom Select options styled styling toggle	Click `#custom-prompt-select` trigger	Dropdown options list renders with custom dark styling	Options list rendered with neon borders and glass container	**PASS**

TC-006	Debate	Limit debate response length parameters	Set length to `short`	Argument text contains between 30 and 40 words	Argument generated under 40 words constraint	**PASS**
TC-007	Debate	Toggle typing animations off	Check Animations = OFF	Arguments are rendered instantly without typing lag	Bubble filled instantly; typing animation skipped	**PASS**
TC-008	Debate	Show and Hide verdict summaries	Close card, then click toggle	Summary card toggles visibility state without error	Summary toggled on and off cleanly	**PASS**

Table 6.1: System Test Scenarios and Results Matrix

6.2 Screenshots

Figure 6.1: Multi-AI Workspace Dashboard Mockup

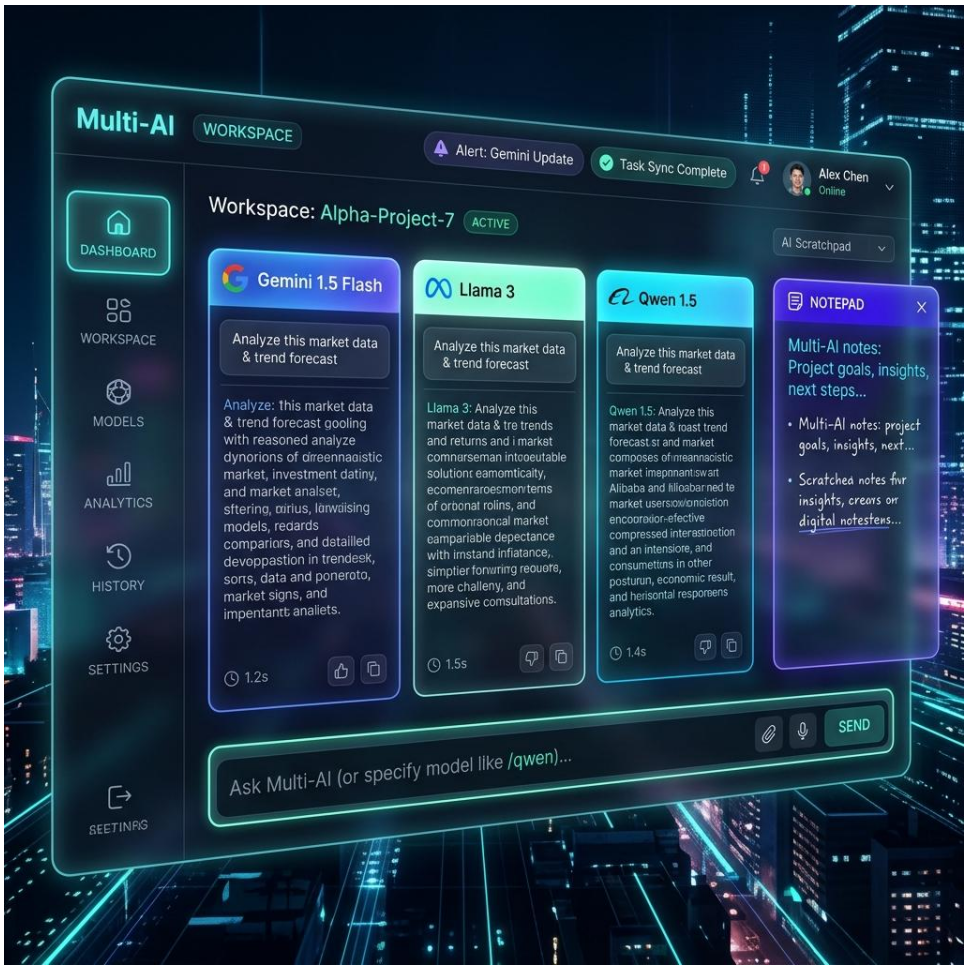


Figure: Multi-AI UI Dashboard Workspace Mockup

CHAPTER 7: CONCLUSION & FUTURE ENHANCEMENTS

7.1 Conclusion

Multi-AI provides a unified, secure workbench for comparing and synthesizing outputs from multiple Large Language Models.

By using parallel asynchronous routing on the backend and Mongoose bulk writes (`bulkWrite`) for database operations, the application prevents data loss and maintains high performance under concurrent write loads.

The fallback architecture ensures the consensus engine remains available even when primary API providers fail. The project demonstrates a production-ready implementation of model orchestration and session management.

7.2 Future Enhancements

- **Stream-Based Response Typing**: Implement Server-Sent Events (SSE) to render model outputs in real-time.
- **Granular Role-Based Access Control (RBAC)**: Add admin dashboards to configure rate limits, monitor system usage, and manage API keys globally.
- **Vector Embeddings Database (RAG Integration)**: Integrate a vector database (such as Pinecone or Milvus) to let users search through their chat history using semantic queries.

REFERENCES

22. **Node.js Documentation**: *Asynchronous Event-Driven Javascript Runtimes*, NodeJS Foundation (https://nodejs.org/docs).
23. **Express.js Guide**: *Routing, Middleware Integration and Rate-Limiting Design* (https://expressjs.com).
24. **Mongoose Documentation**: *Schema Definition, Bulk Writes and Indexing Rules* (https://mongoosejs.com).
25. **Google Gemini API Documentation**: *Generative Language REST Integration Guide* (https://ai.google.dev/docs).
26. **OpenRouter API Specs**: *Unified REST Endpoints for AI Model Selection* (https://openrouter.ai/docs).
27. **LMSYS Chatbot Arena**: *Elo Rating Systems and Comparative Evaluation Methodologies for Large Language Models* (https://chat.lmsys.org).

APPENDIX: SAMPLE SOURCE CODE

Listing 1: Database-Resilient Bulk Write Logic (`backend/routes/workspace.routes.js`)

```
router.put('/history', async (request, response) => {
  try {
    if (!ensureDatabase(response)) return;
    const conversations = Array.isArray(request.body.conversations) ?
request.body.conversations.slice(0, 100) : [];
    const owner = ownerQuery(request);
    const ownerFields = getOwnerFields(request);

    const clientLocalIds = conversations.map(chat => String(chat.id || chat.localId ||
''));

    // 1. Delete conversations no longer present in the client payload
    await Conversation.deleteMany({
      ...owner,
      localId: { $nin: clientLocalIds }
    });

    // 2. Perform bulkWrite upserts for remaining items
    if (conversations.length > 0) {
      const ops = conversations.map(chat => {
        const localId = String(chat.id || chat.localId || '').slice(0, 120);
        return {
          updateOne: {
            filter: { ...owner, localId },
            update: {
              $set: {
                title: String(chat.title || 'Untitled Chat').slice(0, 120),
                isPinned: Boolean(chat.isPinned),
                activeModels: Array.isArray(chat.activeModels) ?
chat.activeModels.slice(0, 10).map(String) : [],
                timestamp: chat.timestamp ? new Date(chat.timestamp) : new Date(),
                messages: chat.messages && typeof chat.messages === 'object' ?
chat.messages : {},
                ...ownerFields
              }
            },
            upsert: true
          }
        };
      });
      await Conversation.bulkWrite(ops);
    }

    const saved = await Conversation.find(owner).sort({ updatedAt: -
1 }).limit(100).lean();
    response.json({ success: true, conversations: saved });
  } catch (error) {
    console.error('History save error:', error);
    response.status(500).json({ success: false, message: 'Failed to save history.' });
  }
});
```

Listing 2: Fallback Logic Engine Implementation (`backend/services/auxiliary.service.js`)

```
import { askGemini } from './gemini.service.js';
import { askOpenRouter } from './openrouter.service.js';
import { askCohere } from './cohere.service.js';
import { askGroq } from './groq.service.js';

export async function askGeminiWithFallback(prompt, fallbackModel = 'google/gemini-
2.5-flash', customKeys = {}) {
  const keys = customKeys || {};
  try {
    const response = await askGemini(prompt, keys.GEMINI_API_KEY);
    if (response === 'Model unavailable') {
      throw new Error('Gemini returned unavailable');
    }
    return { response, fallbackUsed: false };
  } catch (err) {
    console.warn('Gemini request failed, falling back to OpenRouter:', err.message ||
err);
    try {
      const response = await askOpenRouter(prompt, fallbackModel,
keys.OPENROUTER_API_KEY);
      return { response, fallbackUsed: true };
    } catch (fallbackErr) {
      console.error('OpenRouter fallback also failed:', fallbackErr.message ||
fallbackErr);
      try {
        const response = await askCohere(prompt, keys.COHERE_API_KEY);
        if (response && response !== 'Model unavailable') {
          return { response, fallbackUsed: true };
        }
      } catch (cohereErr) {
        console.error('Cohere fallback also failed:', cohereErr.message || cohereErr);
      }

      try {
        const response = await askGroq(prompt, keys.GROQ_API_KEY);
        if (response && response !== 'Model unavailable') {
          return { response, fallbackUsed: true };
        }
      } catch (groqErr) {
        console.error('Groq fallback also failed:', groqErr.message || groqErr);
      }

      return { response: 'Judge service unavailable. Please try again shortly.',
fallbackUsed: true };
    }
  }
}
```